```
            sb.Append(" <strong>OS</strong> ");                          ❽
            sb.Append(                                                 ❽
                _htmlEncoder.Encode(RuntimeInformation.OSDescription)); ❽
        }                                                              ❽
        output.Content.SetHtmlContent(sb.ToString());                  ❾
    }
}
```

❶ Derives from the TagHelper base class

❷ An HtmlEncoder is necessary when writing HTML content to the page.

❸ Decorating properties with HtmlAttributeName allows you to set their values from Razor markup.

❹ The main function called when an element is rendered.

❺ Replaces the <system-info> element with a element

❻ Renders both the   start and end tag

❼ If required, adds a element and the HTML-encoded machine name

❽ If required, adds a element and the HTML-encoded OS name

❾ Sets the inner content of the tag with the HTML-encoded value stored in the string builder

There's a lot of new code in this example, so we'll work through it line by line. First, the class name of the Tag Helper defines the name of the element you must create in your Razor template, with the suffix removed and converted to kebab-case. As this Tag Helper is called `SystemInfoTagHelper`, you must create a `<system-info>` element.

TIP If you want to customize the name of the element, for example to `<env-info>`, but you want to keep the same class name, you can apply `[HtmlTargetElement]` with the desired name, such as `[HtmlTargetElement("Env-Info")]`. HTML tags are not case-sensitive, so you could use `"Env-Info"` or `"env-info"`.

Inject an `HtmlEncoder` into your Tag Helper so you can HTML-encode any data you write to the page. As you saw in chapter 29, you should always